

0806_이신영

motion in-betweening 논문 분석

1. 초기 RNN 계열

Zhang & van de Panne, 2018 — "Data-driven Autocompletion for Keyframe Animation"

- 새로 도입한 것: 딥러닝 기반 in-betweening의 초기 시도. 단순한 2D 캐릭터를 대상으로 RNN이 키프레임 사이를 자동완성
- 아키텍처: 단순 RNN

Harvey & Pal, 2018 — "Recurrent Transition Networks (RTN)"

- 새로 도입한 것: 더 복잡한 인간 모션까지 확장. 미래 정보(목표 키프레임)를 다루기 위해 두 개의 별도 인코더(target encoder + offset encoder)를 쓰는 구조를 처음 제안
- 아키텍처: LSTM 기반, 고정된 60프레임 길이만 지원

Zhou et al., 2020 — "Generative Tweening"

- 새로 도입한 것: RTN을 확장해서 1프레임부터 수천 프레임까지 가변 길이 in-betweening 지원. Adversarial training(GAN 방식) 도입
- 아키텍처: RTN 확장 + discriminator

2. 벤치마크 확립 — 이후 모든 연구의 기준점

★ Harvey, Yurick, Nowrouzezahrai, Pal, 2020 — "Robust Motion In-betweening (RMIB)"

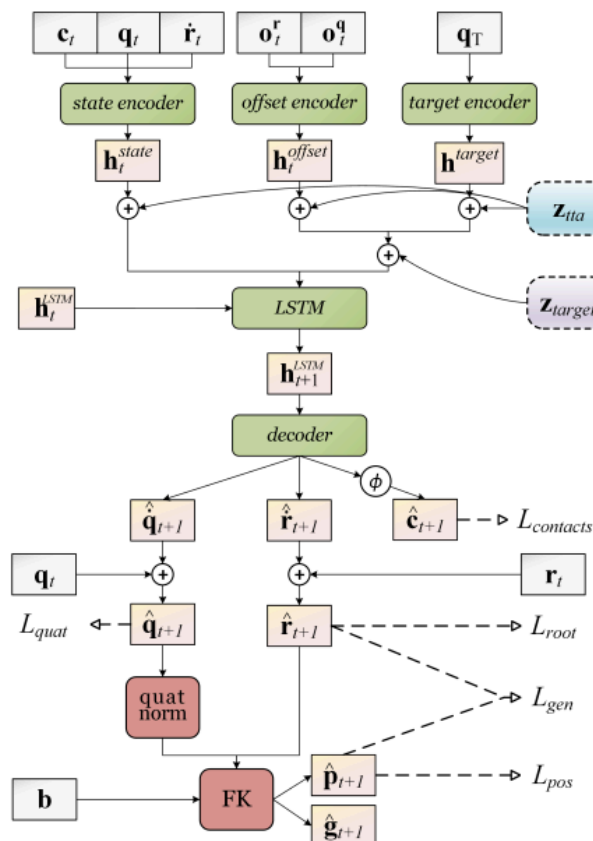
▼ Robust Motion In-betweening

문제의식

- 기존 최고 성능의 모션 예측 모델(과거만 보고 미래를 예측하는 모델)에 "미래 목표 키프레임" 조건 정보만 단순히 추가해서는 견고한 transition generator로 못 바꿈
- 조건을 어떻게 주느냐가 핵심 문제

아키텍처 상세:

- **Offset encoder:** "지금 자세"와 "목표 키프레임" 사이의 차이(root 위치·방향의 선형 차이, 관절별 로컬 quaternion 차이)를 입력으로 받는 인코더
- **Target encoder:** 목표 키프레임 자체를 인코딩하는 별도 인코더
- 이 두 인코더의 임베딩이 **LSTM** 레이어로 들어가서 시간적 동역학을 모델링
- LSTM 출력이 feed-forward 디코더를 거쳐 최종 자세와 발 접촉 여부를 예측
- **손실 함수 4종:** quaternion loss(회전), position loss(forward kinematics로 구한 전역 위치), foot contact loss, adversarial loss(GAN)



핵심 기여

- **ztta (time-to-arrival embedding)**: 목표까지 몇 프레임 남았는지를 sinusoidal 형태로 인코딩해서 각 타임스텝의 latent 표현에 더해줌. 이게 없으면 전환 끝부분에서 부자연스럽고, ztta를 추가하면 훨씬 부드러워짐
- **ztarget (scheduled target noise)**: 목표 자체에 학습 중 노이즈를 섞어서 다양한 변주를 생성할 수 있게 함(같은 시작-끝 조건에서도 100프레임 전환을 10번 다르게 샘플링할 수 있음을 보임)

우리 모델에 가져올 부분:

- **ztta(time-to-arrival) 임베딩**
 - L=5/10/20/30을 따로 모델을 만들지 말고, "가려진 지 몇 프레임째인지"를 이 방식으로 인코딩하면 하나의 모델로 통합 가능
- Position loss
 - "관절별 절대 위치"와 "관절 간 상대적 회전/오프셋"을 나눠서 손실을 설계
 - 지금은 MPJPE 하나만 쓰는데, 손가락 관절 간 상대 위치를 따로 loss에 반영하면 뼈 길이가 안 어긋나는 효과를 기대)
- 새로 도입한 것
 - LaFAN1이라는 표준 벤치마크 데이터셋을 이 논문이 직접 만들어서 공개
 - 이후 거의 모든 후속 연구가 이 데이터셋과 이 논문의 평가지표(L2P, L2Q, NPSS)를 그대로 씀
 - **Time-to-arrival(tta) 임베딩**(목표까지 남은 시간을 sinusoidal 인코딩)과 **scheduled target noise**(학습 중 목표에 노이즈를 섞어 견고성 확보)를 도입
- 아키텍처: LSTM + adversarial training

Kaufmann et al., 2020 — "Convolutional Autoencoders for Human Motion Infilling"

- 새로 도입한 것: RNN이 아닌 **컨볼루션 기반**의 첫 시도, end-to-end 학습 가능한 오토 인코더
- 아키텍처: Convolutional autoencoder

3. Transformer 기반 모델

★ Kim et al., 2022 — "Conditional Motion In-betweening (CMIB)"

▼ Conditional Motion In-betweening

문제의식

- RNN 기반 접근은 긴 시퀀스에서 오차가 누적되는 문제와 병렬화가 어렵다는 근본적 한계
- Transformer의 self-attention으로 이 문제를 풀려는 흐름의 연장선에서 나온 논문

아키텍처

- 시작-끝 키프레임을 ****선형보간(LERP)****으로 채운 전체 시퀀스를 Transformer 인코더에 입력하고, self-attention으로 전체 시퀀스를 한 번에 처리해서 자연스러운 중간 프레임들을 출력.
- "빠진 부분을 마스킹하고 그 자리를 채운다"는 점에서 BERT의 masked language modeling과 구조적으로 유사한 첫 시도.

우리 모델에 가져올 부분:

- 우리 bidirectional 셋업(before/gap/after)에 이 프레임이 정확히 대응
 - **gap 구간을 어떤 값으로 채워서 입력할지**(우리는 SILK 발견에 따라 0-채우기를 기본으로 하되, 이 논문 방식인 LERP-채우기도 ablation으로 비교)
- semantic conditioning(제스처 타입 같은 의미 정보로 조건 걸기)
 - "이 손이 어떤 제스처를 하고 있었는지" 같은 부가 정보를 조건으로 줄 수 있음

• 새로 도입한 것

- **최초로 in-betweening을 BERT 같은 masked motion modeling 문제로 재정**의한 Transformer 논문
- 빈 프레임을 시작-끝 키프레임 사이 **선형보간(LERP)값으로 미리 채워서 입력**하는 방식을 처음 제안
 - 이후 대부분의 후속 연구가 이 방식을 그대로 따름

- 제스처 타입 같은 의미 정보(semantic conditioning)로도 조건을 걸 수 있게 함
- **아키텍처:** Transformer, seq2seq

Duan et al., 2022 — "A Unified Framework for Real Time Motion Completion"

- **새로 도입한 것:** CMIB 확장. **키프레임/빈 프레임/무시할 프레임을 구분하는 학습 가능한 임베딩**을 추가하고, Transformer 앞뒤에 컨볼루션 레이어를 붙여 시간 구조를 보강
- **아키텍처:** Conv + Transformer + Conv

Oreshkin et al., 2023 — "Motion Inbetweening via Deep Δ -Interpolator (Delta-Interpolator)"

- **새로 도입한 것:** 보간값을 입력으로 안 주고, **모델이 "보간값 대비 잔차(residual)"만 예측**하도록 설계 전환. 모델이 선형보간과 실제 모션 두 가지를 동시에 배워야 하는 구조
- **아키텍처:** 기존 키프레임용/빈 프레임용 인코더 2개 분리

Qin, Zheng, Zhou, 2022 — "Motion In-betweening via Two-stage Transformers"

▼ Two-stage Transformers

아키텍처:

- **Context Transformer:** 전체 전환의 "큰 그림"(대략적인 궤적, 저주파 성분)을 먼저 예측
- **Detail Transformer:** Context Transformer의 결과를 입력받아서 미세한 디테일(고주파 성분)을 정교하게 다듬음
- 두 단계 모두 encoder-only Transformer이고, 보간된 입력이 필요 없는 구조 (CMIB와 달리 빈 곳을 미리 채우지 않고 Transformer가 처음부터 생성)

우리 모델에 가져올 부분:

- **손가락 움직임에 특히 잘 맞을 수 있는 아이디어**
 - 손 전체의 대략적인 궤적(예: 주먹을 쥐는 큰 흐름)을 먼저 잡고, 그 다음 각 손가락의 미세한 타이밍 차이(디테일)를 다듬는 2단계 구조가 손의 계층적 특성(손목 움직임이 크고, 손가락 끝 움직임이 미세함)과 자연스럽게 맞아떨어짐

- 다만 2단계 구조는 구현·학습 복잡도가 올라감
- 새로 도입한 것: **Context Transformer**(전체적인 전환의 큰 그림을 먼저 잡음) + **Detail Transformer**(그 결과를 정교하게 다듬음) 2단계 구조. 보간 입력이 아예 필요 없어짐. LaFAN1 벤치마크에서 오랫동안 최고 성능을 유지한 논문
- **아키텍처**: Encoder-only Transformer 2개, 2단계 학습

4. Skeleton 구조를 명시적으로 반영하는 계열

여러 연구(Xu et al. 2024 DanceCraft, Agrawal et al. 2024 Skel-betweenner, Xu et al. 2024 STG)

- 새로 도입한 것: 관절들이 트리 구조(팔 → 어깨, 다리 → 골반 등)로 연결돼 있다는 걸 명시적으로 인코더/디코더 설계에 반영(skeleton-aware architecture, skeletal graph transformer, graph U-Net 등)
- **우리 프로젝트 관점**: 손도 손목 → 손가락으로 뻗는 트리 구조라 이 아이디어가 유효할 수 있지만, 논문들이 몸 전체(팔다리 몇 개)를 대상으로 한 거라 손(21개 관절, 훨씬 조밀)에 그대로 적용 가능한지는 검증이 필요

Wang, Ge, Lu, 2025 — "ST-TransMIB"

- 새로 도입한 것: 프레임 내 관절 간 상호작용을 다루는 **공간(spatial) transformer** + 전역 흐름과 세부 변화를 동시에 잡는 **다중 스케일 시간(temporal) transformer** 결합
- **아키텍처**: Spatial + Multi-scale Temporal Transformer

5. 최신 흐름

SILK, 2025

▼ SILK

아키텍처

- 단일 Transformer 인코더(6 layer, 8 head 정도의 표준 구성)
- 별도의 target/offset 인코더나 2단계 구조 없이 **하나의 인코더로 전체를 처리**

6가지 핵심 설계 선택

1. 데이터를 촘촘하게 샘플링해서 양을 늘림
2. Root-space(전역 기준) 좌표 표현
3. 속도(velocity) 정보를 입력에 추가
4. 빈 프레임은 0으로 채움 (LERP보다 나음)
5. 단순한 상대적 위치 인코딩만 사용 (keyframe 전용 임베딩 불필요)
6. 단일 L1 손실

우리 모델에 가져올 부분 — 사실상 우리 베이스 아키텍처로 채택:

- 이게 제일 단순하면서 검증된 구조라, **우리 모델의 뼈대로 그대로 채택**하는 게 합리적
- 다만 5번(위치 인코딩)은 RMIB의 ztta와 상충하는 부분이 있어서, 우리는 "L을 통합 처리하고 싶다"는 목표가 있으니 **SILK의 단순함 + RMIB의 ztta**를 결합하는 게 우리만의 조합이 될 수 있음
- Root-space 표현은 우리 식으로 바꾸면 "손목 기준 상대좌표"(이미 우리가 쓰고 있는 정규화 방식과 유사한 발상)

• 새로 도입한 것

- 위 모든 복잡한 설계(다중 인코더, skeleton-aware, 2단계 학습)가 사실 필요 x 입증
- **0-채우기, root-space 좌표 표현, velocity 입력, 촘촘한 데이터 샘플링**이 아키텍처 복잡도보다 중요하다는 걸 보임
- **아키텍처**: 단일 Transformer 인코더 (6 layer, 8 head)